

Solutions for 50 Java Streams Practice Questions for SDETs

Introduction

This document contains detailed solutions for all 50 Java Streams practice questions.

Each solution includes:

- Java Stream solution
- Step-by-step explanation
- Why a particular Stream method is used
- Real SDET relevance
- Beginner-friendly understanding

The solutions are arranged in the exact same order as the practice question document.

Level 1 — Beginner (Questions 1–15)

Question 1 — Print All Browser Names

Solution

```
List<String> browsers = Arrays.asList("Chrome", "Firefox", "Edge");  
  
browsers.stream()  
    .forEach(System.out::println);
```

Explanation

Step 1

```
browsers.stream()
```

Converts list into Stream.

Step 2

```
forEach(System.out::println)
```

Prints each element.

Why This Solution?

- `forEach()` is used when we want to iterate through all elements.
 - Streams provide cleaner iteration than loops.
-

Question 2 — Print All Even Numbers

Solution

```
List<Integer> nums = Arrays.asList(1,2,3,4,5,6);  
  
nums.stream()  
    .filter(n -> n % 2 == 0)  
    .forEach(System.out::println);
```

Explanation

`filter()`

```
n -> n % 2 == 0
```

Keeps only even numbers.

Why This Solution?

`filter()` is specifically used to keep matching elements.

Question 3 — Print All Odd Numbers

Solution

```
nums.stream()  
    .filter(n -> n % 2 != 0)  
    .forEach(System.out::println);
```

Explanation

- `% 2 != 0` checks odd numbers.
 - `filter()` removes unwanted numbers.
-

Question 4 — Convert Browser Names to Uppercase

Solution

```
List<String> browsers = Arrays.asList("chrome", "firefox");  
  
browsers.stream()  
    .map(String::toUpperCase)  
    .forEach(System.out::println);
```

Explanation

map()

Used when transforming data.

Here:

```
chrome → CHROME
```

Why map()?

Because we are transforming each element.

Question 5 — Multiply Numbers by 10

Solution

```
List<Integer> nums = Arrays.asList(1,2,3);

nums.stream()
    .map(n -> n * 10)
    .forEach(System.out::println);
```

Explanation

`map()` changes every number.

Example:

```
1 → 10
2 → 20
```

Question 6 — Count Total Elements

Solution

```
long count = nums.stream()
    .count();

System.out.println(count);
```

Explanation

`count()` returns total elements in Stream.

Question 7 — Count Failed Test Cases

Solution

```
List<String> status = Arrays.asList("PASS", "FAIL", "PASS", "FAIL");

long failedCount = status.stream()
    .filter(s -> s.equals("FAIL"))
    .count();

System.out.println(failedCount);
```

Explanation

Step 1

Filter FAIL status.

Step 2

Count remaining elements.

SDET Relevance

Very common in automation reporting.

Question 8 — Sort Numbers in Ascending Order

Solution

```
List<Integer> nums = Arrays.asList(5,1,4,2);

nums.stream()
    .sorted()
    .forEach(System.out::println);
```

Explanation

`sorted()` sorts elements naturally.

Question 9 — Sort Browser Names Alphabetically

Solution

```
browsers.stream()  
    .sorted()  
    .forEach(System.out::println);
```

Explanation

Strings are sorted alphabetically.

Question 10 — Remove Duplicate Numbers

Solution

```
List<Integer> nums = Arrays.asList(1,1,2,2,3,3);  
  
nums.stream()  
    .distinct()  
    .forEach(System.out::println);
```

Explanation

`distinct()` removes duplicate elements.

Question 11 — Remove Duplicate Browser Names

Solution

```
List<String> browsers = Arrays.asList("Chrome", "Chrome", "Edge");  
  
browsers.stream()  
    .distinct()  
    .forEach(System.out::println);
```

Explanation

Only unique values remain.

Question 12 — Print First 3 Elements

Solution

```
nums.stream()  
    .limit(3)  
    .forEach(System.out::println);
```

Explanation

`limit(3)` keeps only first 3 elements.

Question 13 — Skip First 2 Elements

Solution

```
nums.stream()  
    .skip(2)  
    .forEach(System.out::println);
```

Explanation

`skip(2)` ignores first 2 elements.

Question 14 — Find Names Starting with A

Solution

```
List<String> names = Arrays.asList("Amit", "John", "Alex");  
  
names.stream()
```

```
.filter(name -> name.startsWith("A"))  
.forEach(System.out::println);
```

Explanation

startsWith("A")

Checks whether name begins with A.

Question 15 — Find Numbers Greater Than 50

Solution

```
nums.stream()  
    .filter(n -> n > 50)  
    .forEach(System.out::println);
```

Explanation

`filter()` keeps numbers greater than 50.

Level 2 — Beginner to Intermediate (Questions 16–30)

Question 16 — Collect Even Numbers into List

Solution

```
List<Integer> evenNums = nums.stream()  
    .filter(n -> n % 2 == 0)  
    .collect(Collectors.toList());
```


Explanation

`collect()`

Used to store Stream result.

Why `collect()`?

Because we want final result as List.

Question 17 — Collect Failed Test Cases

Solution

```
List<String> failed = status.stream()
    .filter(s -> s.equals("FAIL"))
    .collect(Collectors.toList());
```

Explanation

Stores all FAIL statuses into separate list.

Question 18 — Check if Any Test Failed

Solution

```
boolean result = status.stream()
    .anyMatch(s -> s.equals("FAIL"));

System.out.println(result);
```

Explanation

`anyMatch()` returns true if ANY element matches condition.

Question 19 — Check if All Tests Passed

Solution

```
boolean result = status.stream()
    .allMatch(s -> s.equals("PASS"));
```

Explanation

`allMatch()` checks whether ALL elements match.

Question 20 — Check if No Negative Numbers Exist

Solution

```
boolean result = nums.stream()
    .noneMatch(n -> n < 0);
```

Explanation

`noneMatch()` ensures no element matches condition.

Question 21 — Find First Failed Test

Solution

```
Optional<String> failed = status.stream()
    .filter(s -> s.equals("FAIL"))
    .findFirst();

failed.ifPresent(System.out::println);
```

Explanation

`findFirst()`

Returns first matching element.

Optional

Used because result may or may not exist.

Question 22 — Find Any Browser Name

Solution

```
Optional<String> browser = browsers.stream()  
                                .findAny();
```

Explanation

`findAny()` returns any available element.

Question 23 — Sum All Numbers

Solution

```
int sum = nums.stream()  
               .reduce(0, Integer::sum);  
  
System.out.println(sum);
```

Explanation

`reduce()`

Combines multiple values into single value.

Breakdown

0

Initial value.

`Integer::sum`

Adds numbers together.

Question 24 — Find Maximum Number

Solution

```
Optional<Integer> max = nums.stream()  
    .max(Integer::compareTo);
```

Explanation

`max()` finds largest element using comparator.

Question 25 — Find Minimum Number

Solution

```
Optional<Integer> min = nums.stream()  
    .min(Integer::compareTo);
```

Explanation

`min()` finds smallest element.

Question 26 — Convert List to Set

Solution

```
Set<Integer> set = nums.stream()  
                        .collect(Collectors.toSet());
```

Explanation

Set automatically removes duplicates.

Question 27 — Join Browser Names

Solution

```
String result = browsers.stream()  
                      .collect(Collectors.joining(", "));  
  
System.out.println(result);
```

Explanation

`joining()` combines strings.

Question 28 — Count Total Failed Tests

Solution

```
long count = status.stream()  
                  .filter(s -> s.equals("FAIL"))  
                  .count();
```

Explanation

Very common in reporting dashboards.

Question 29 — Convert Names to Lowercase

Solution

```
names.stream()  
    .map(String::toLowerCase)  
    .forEach(System.out::println);
```

Explanation

Transforms names into lowercase.

Question 30 — Filter Mobile Numbers Starting with 9

Solution

```
numbers.stream()  
    .filter(num -> num.startsWith("9"))  
    .forEach(System.out::println);
```

Explanation

Filters numbers beginning with 9.

Level 3 — Intermediate (Questions 31–40)

Question 31 — Flatten Nested List

Solution

```
List<List<Integer>> data = Arrays.asList(  
    Arrays.asList(1,2),  
    Arrays.asList(3,4)
```

```
);  
  
List<Integer> result = data.stream()  
    .flatMap(List::stream)  
    .collect(Collectors.toList());
```

Explanation

flatMap()

Converts:

```
[[1,2],[3,4]]
```

Into:

```
[1,2,3,4]
```

Question 32 — Flatten Multiple Test Data Sheets

Solution

```
sheets.stream()  
    .flatMap(List::stream)  
    .forEach(System.out::println);
```

Explanation

Useful for combining multiple data sources.

Question 33 — Group Employees by Department

Solution

```
Map<String, List<Employee>> grouped =  
employees.stream()  
    .collect(Collectors.groupingBy(Employee::getDepartment));
```

Explanation

`groupingBy()` groups elements by condition.

Question 34 — Group Test Cases by Status

Solution

```
Map<String, List<TestCase>> grouped =  
testCases.stream()  
    .collect(Collectors.groupingBy(TestCase::getStatus));
```

Explanation

Useful in reporting.

Example:

```
PASS → all passed tests  
FAIL → all failed tests
```

Question 35 — Partition Numbers into Even and Odd

Solution

```
Map<Boolean, List<Integer>> result =  
  nums.stream()  
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));
```

Explanation

partitioningBy() creates:

```
true → even  
false → odd
```

Question 36 — Partition Test Results into Pass and Fail

Solution

```
Map<Boolean, List<TestResult>> result =  
  results.stream()  
    .collect(Collectors.partitioningBy(TestResult::isPassed));
```

Explanation

Splits results into:

- passed
- failed

Question 37 — Sort Employees by Salary

Solution

```
employees.stream()  
    .sorted(Comparator.comparing(Employee::getSalary))  
    .forEach(System.out::println);
```

Explanation

Comparator defines sorting logic.

Question 38 — Find Highest Salary Employee

Solution

```
Optional<Employee> emp = employees.stream()  
    .max(Comparator.comparing(Employee::getSalary));
```

Explanation

Finds employee having highest salary.

Question 39 — Extract Employee Names

Solution

```
employees.stream()  
    .map(Employee::getName)  
    .forEach(System.out::println);
```

Explanation

Extracts only required field.

Question 40 — Chain Multiple Stream Operations

Solution

```
List<Integer> result = nums.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * 10)
    .sorted(Comparator.reverseOrder())
    .collect(Collectors.toList());
```

Explanation

Step 1

Filter even numbers.

Step 2

Multiply by 10.

Step 3

Sort descending.

Step 4

Collect into List.

Level 4 — Advanced (Questions 41-50)

Question 41 — Find Duplicate Elements

Solution

```
Set<Integer> unique = new HashSet<>();

nums.stream()
```

```
.filter(n -> !unique.add(n))  
.forEach(System.out::println);
```

Explanation

Logic

`Set.add()` returns:

- true → new element
- false → duplicate

Question 42 — Find Second Highest Number

Solution

```
Optional<Integer> secondHighest = nums.stream()  
    .distinct()  
    .sorted(Comparator.reverseOrder())  
    .skip(1)  
    .findFirst();
```

Explanation

Step 1

Remove duplicates.

Step 2

Sort descending.

Step 3

Skip highest number.

Step 4

Get next number.

Question 43 — Find Longest String

Solution

```
Optional<String> longest = names.stream()  
                                .max(Comparator.comparing(String::length));
```

Explanation

Comparator compares string lengths.

Question 44 — Count Occurrences of Each Word

Solution

```
Map<String, Long> result = words.stream()  
                                .collect(Collectors.groupingBy(  
                                    word -> word,  
                                    Collectors.counting()  
                                ));
```

Explanation

groupingBy()

Groups same words.

counting()

Counts frequency.

Question 45 — Validate All API Status Codes are 200

Solution

```
boolean result = statusCodes.stream()
    .allMatch(code -> code == 200);
```

Explanation

Very common API validation scenario.

Question 46 — Find ERROR Logs from Large Log File

Solution

```
logs.stream()
    .filter(log -> log.contains("ERROR"))
    .forEach(System.out::println);
```

Explanation

Filters only ERROR logs.

Question 47 — Process Logs Using Parallel Stream

Solution

```
long count = logs.parallelStream()
    .filter(log -> log.contains("ERROR"))
    .count();
```

Explanation

parallelStream()

Processes large data in parallel.

Important Note

Useful for:

- large files
 - heavy data processing
-

Question 48 — Extract Unique Failed Test Names

Solution

```
failedTests.stream()  
    .distinct()  
    .forEach(System.out::println);
```

Explanation

Removes duplicate failed tests.

Question 49 — Create Map of Employee Name and Salary

Solution

```
Map<String, Integer> map = employees.stream()  
    .collect(Collectors.toMap(  
        Employee::getName,  
        Employee::getSalary  
    ));
```

Explanation

toMap()

Creates key-value structure.

Final Structure

Name → Salary

Question 50 — Real SDET End-to-End Scenario

Solution

```
List<String> result = failedTests.stream()
                                .distinct()
                                .map(String::toUpperCase)
                                .sorted()
                                .collect(Collectors.toList());
```

Explanation

Step 1 — distinct()

Removes duplicates.

Step 2 — map()

Converts to uppercase.

Step 3 — sorted()

Sorts alphabetically.

Step 4 — collect()

Stores final result.

Why This Question is Important?

This resembles real automation framework logic where:

- failed tests are processed
 - duplicates removed
 - reports normalized
 - data sorted before reporting
-

Final Advice for Students

Streams become easy when you:

1. Understand data flow
 2. Learn one operation at a time
 3. Practice real scenarios
 4. Read stream pipeline step-by-step
-

Most Important Stream Methods for SDETs

Method	Importance
filter()	Very High
map()	Very High
collect()	Very High
anyMatch()	High
groupingBy()	High
flatMap()	High
reduce()	High
distinct()	Very High
sorted()	High
Optional	High

Final Goal

After practicing these solutions, students should be able to:

- write cleaner automation code
- process API data efficiently
- analyze logs using Streams
- work with Selenium collections
- build reporting utilities
- solve Stream interview questions confidently